DOCUMENT TECHNIQUE

Conception d’une application en architecture microservices

Titre du projet : SafePay

Membres du groupe :

Rahmouni Yassmine,

Kobbi Yosr,

Mhiri Mohamed Fedi,

Bougacha Mohamed Aziz

Encadrante : Ben Salah Ons

Année universitaire : 2026/2027



Table des matières

Table des matières **2**
1. Description du sujet **3**
1.1 Contexte et problématique 3
1.2 Description du projet 3
1.3 Objectifs du projet 3
1.4 Justification du choix des microservices 4
2. Acteurs et fonctionnalités **4**
2.1 Acteurs 4
2.2 Fonctionnalités par acteur 5
2.3 Diagramme de cas d'utilisation 6
3. Découpage en microservices **7**
3.1 Critères de découpage 7
3.2 Liste des microservices 8
Composants d'infrastructure : 9
3.3 Description détaillée de chaque service 9
MS1 – User 9
MS2 – Wallet 9
MS3 – Payment 10
MS4 – Card 10
MS5 – Fraud Detection (conception commune de l'équipe) 11
4. Données gérées par chaque service **11**
4.1 Vue d'ensemble 11
4.2 Modèle de données par service 12
MS1 – User (DB_USER, H2) 12
4.3 Gestion de la cohérence des données 16
5. Interactions entre les services **17**
5.1 Communications synchrones 17
5.2 Communications asynchrones (RabbitMQ) 17
5.3 Scénarios d'interaction 18
6. Diagramme d'architecture globale **18**
Légende et explications 19
7. Autres éléments de conception **19**
7.1 Technologies utilisées 19
7.2 API Gateway et découverte de services 20
7.3 Sécurité 20
7.4 Déploiement et conteneurisation 20
7.5 Supervision et journalisation 20
7.6 Gestion des erreurs et résilience 21
7.7 Limites et améliorations futures 21
8. Conclusion **21**

1. Description du sujet

1.1 Contexte et problématique

Avec le développement des technologies numériques, les paiements électroniques et les transferts d'argent en ligne sont devenus de plus en plus courants, offrant aux utilisateurs des services rapides et accessibles.

Cependant, cette évolution entraîne également des risques liés à la cybersécurité, tels que les fraudes bancaires, les transactions non autorisées et le phishing. Face à ces menaces, les méthodes de sécurité classiques ne permettent pas toujours de détecter rapidement les opérations suspectes.

Il devient donc essentiel de développer des solutions intelligentes capables d'analyser les transactions, d'identifier les comportements inhabituels et de prévenir les activités frauduleuses.

Problématique : Comment améliorer la détection et la prévention de la fraude dans les services financiers numériques ?

C'est dans ce contexte que s'inscrit SafePay, une plateforme visant à renforcer la sécurité des paiements numériques grâce à un système intelligent de détection des fraudes.

1.2 Description du projet

SafePay est une plateforme de paiement numérique sécurisée qui vise à faciliter et à sécuriser les opérations financières des utilisateurs à travers un environnement digital centralisé.

La plateforme permet aux utilisateurs de créer et de gérer leurs comptes, de disposer de portefeuilles électroniques (Wallets), d'effectuer des paiements en ligne, de transférer de l'argent et de gérer leurs cartes virtuelles.

SafePay met également l'accent sur la sécurité des transactions financières grâce à un système intelligent de détection des fraudes. Ce dernier analyse les transactions en fonction de différents critères afin d'identifier les comportements inhabituels et les activités potentiellement frauduleuses. Lorsqu'une transaction est considérée comme suspecte, elle peut être signalée et mise en attente pour vérification avant son traitement définitif.

1.3 Objectifs du projet

- *Les principaux objectifs de SafePay sont :*
  - *Simplifier les paiements numériques et les transferts d'argent.*
  - *Assurer une gestion efficace des portefeuilles électroniques et des soldes.*
  - *Permettre la création et la gestion de cartes virtuelles.*
  - *Garantir la sécurité des opérations financières.*
  - *Détecter les transactions suspectes et prévenir les tentatives de fraude.*
  - *Améliorer l'expérience utilisateur grâce à une plateforme accessible et intuitive.*

1.4 Justification du choix des microservices

Pourquoi une architecture microservices plutôt qu'un monolithe ?

Le choix d'une architecture microservices pour SafePay, plutôt qu'une architecture monolithique, s'explique par la nécessité de séparer les différentes fonctionnalités de la plateforme en services indépendants.

Cette architecture présente plusieurs avantages :

- **Scalabilité :** chaque microservice peut être dimensionné indépendamment selon sa charge. Par exemple, le Payment Service peut être renforcé lorsque le nombre de transactions augmente.
- **Indépendance :** chaque service possède ses propres responsabilités et peut évoluer sans modifier l'ensemble de l'application.
- **Déploiement indépendant :** les mises à jour et les corrections peuvent être déployées service par service, sans redéployer toute la plateforme.
- **Maintenance facilitée :** la séparation des fonctionnalités permet de simplifier le développement, les tests et la maintenance.
- **Flexibilité technologique :** chaque microservice peut utiliser les technologies les plus adaptées à ses besoins.

Ce choix est particulièrement intéressant pour SafePay, car il permet de faire évoluer indépendamment les services de paiement et de détection de fraude, tout en facilitant l'intégration de nouvelles fonctionnalités basées sur l'intelligence artificielle

2. Acteurs et fonctionnalités

2.1 Acteurs

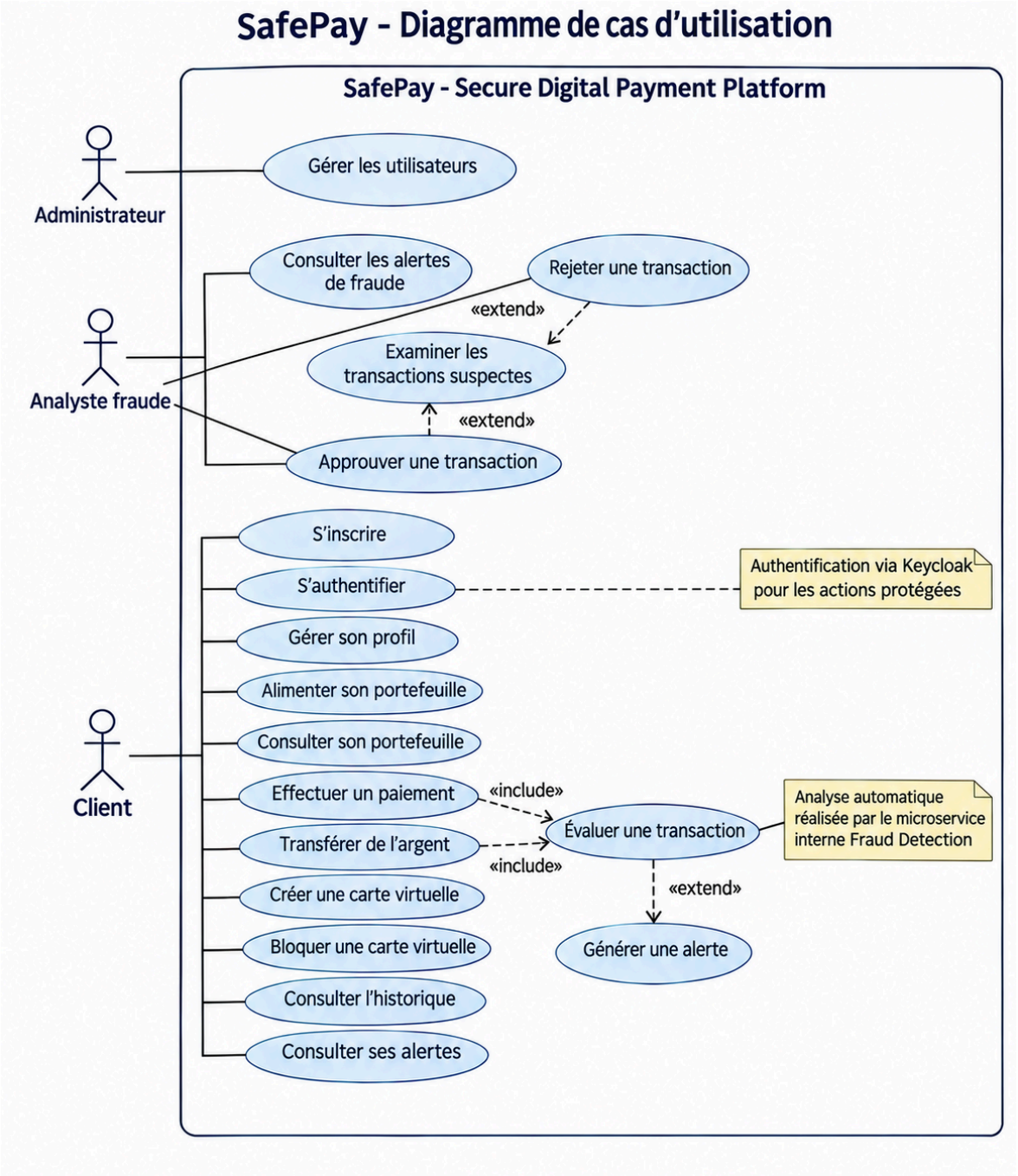
Acteur	Type	Description / rôle
Client	Humain	Utilisateur final de la plateforme. Il gère son compte, son portefeuille et ses cartes virtuelles, et effectue des paiements et des transferts.
analyste fraude	Humain	Responsable de la supervision. Il examine les transactions signalées comme suspectes et consulte les alertes.
Fraud Detection	acteur système	Acteur automatique. Il évalue chaque transaction en temps réel et lève une alerte lorsqu'elle est jugée suspecte.
Admin	Humain	Responsable de la gestion des comptes. Il gère les utilisateurs (suspension, réactivation, décision KYC) et consulte les alertes.

2.2 Fonctionnalités par acteur

Acteur	Fonctionnalité	Description	Priorité
Client	S'authentifier	Permettre au client d'accéder à la plateforme SafePay de manière sécurisée en saisissant ses identifiants	Haute
Client	S'inscrire	Créer un compte avec ses informations personnelles et s'authentifier.	Haute
Client	Gérer son profil	Consulter et modifier ses informations personnelles et ses paramètres.	Moyenne
Client	Alimenter son portefeuille	Ajouter de l'argent au portefeuille (recharge).	Haute
Client	Consulter son portefeuille	Voir le solde et l'état du portefeuille.	Haute
Client	Effectuer un paiement	Régler un achat ou un service à partir de son portefeuille ou d'une carte virtuelle.	Haute
Client	Transférer de l'argent	Envoyer de l'argent à un autre utilisateur.	Haute
Client	Créer une carte virtuelle	Générer une nouvelle carte virtuelle liée à son portefeuille.	Moyenne
Client	Bloquer une carte virtuelle	Désactiver une carte en cas de risque ou de perte.	Haute
Client	Consulter l'historique	Voir la liste de ses transactions passées.	Moyenne
Client	Consulter les alertes	Voir les alertes de sécurité ou de fraude le concernant.	Moyenne
analyste fraude	Examiner les transactions signalées	Consulter le détail des transactions marquées comme suspectes.	Haute
analyste fraude	Approuver ou rejeter une transaction	Décider du sort d'une transaction signalée après analyse.	Haute
Administrateur	Gérer les utilisateurs	Consulter, activer, suspendre ou bannir des comptes.	Moyenne
analyste fraude	Consulter les alertes	Suivre toutes les alertes levées sur la plateforme.	Moyenne
Fraud Detection	Évaluer chaque transaction	Analyser chaque transaction (montant, fréquence, comportement) pour calculer un niveau de risque.	Haute

Acteur	Fonctionnalité	Description	Priorité
Fraud Detection	Lever des alertes	Générer une alerte et signaler la transaction à l'analyste lorsqu'un risque est détecté.	Haute

2.3 Diagramme de cas d'utilisation



3. Découpage en microservices

3.1 Critères de découpage

*Le découpage de SafePay suit la règle **« une capacité métier = un service »**, selon les contextes bornés (bounded contexts) du Domain-Driven Design. Chaque service regroupe un ensemble cohérent de règles et de données, et les échanges entre services passent par des API REST ou des événements, jamais par un accès direct à la base d'un autre service.*

Les critères suivants ont guidé le découpage :

- ***Capacité métier distincte :** chaque service répond à un besoin précis (identité, solde, paiement, carte, fraude) avec son propre vocabulaire et ses propres règles.*
- ***Propriété des données :** chaque donnée a un seul service propriétaire. Par exemple, seul Wallet écrit les soldes.*
- ***Faible couplage :** les services communiquent par des contrats clairs (endpoints et événements), ce qui permet de modifier l'un sans impacter les autres.*
- ***Forte cohésion :** les fonctionnalités qui évoluent ensemble restent dans le même service (par exemple le blocage et le déblocage d'une carte dans Card).*
- ***Autonomie de déploiement et d'équipe :** chaque service peut être développé, testé et déployé séparément, avec un responsable par service.*
- ***Isolation des pannes :** la défaillance d'un service ne doit pas bloquer les autres. Par exemple, si Fraud est indisponible, le paiement reste en attente au lieu d'être approuvé par défaut.*
- ***Adéquation technologique :** chaque service utilise le SGBD qui convient à son besoin (H2, MySQL, PostgreSQL avec JSONB pour les preuves antifraude).*
- ***Sécurité et séparation des responsabilités :** l'authentification est confiée à Keycloak, le profil à User, et la décision de fraude à un service dédié.*

Service	Capacité métier	Données dont il est propriétaire
User	Profil, KYC, rôles, état du compte	Utilisateurs
Wallet	Soldes et mouvements	Portefeuilles, transactions de portefeuille
Payment	Paiements, transferts, statuts	Paiements et historique des statuts
Card	Cartes virtuelles et plafonds	Cartes et historique des statuts
Fraud	Score de risque, alertes, revue	Contrôles de fraude et alertes

3.2 Liste des microservices

Microservice	Responsabilité	Technologie	Port
MS1 – User	Gère le cycle de vie du compte client : création et mise à jour du profil, suivi du statut KYC (vérification d'identité), attribution des rôles (CLIENT, ADMIN, FRAUD_ANALYST) et activation ou suspension du compte. Fournit aux autres services l'état du client. Délègue l'authentification et l'émission des JWT à Keycloak.	Spring Boot	8081
MS2 – Wallet	Gère le portefeuille de chaque client : création, consultation du solde et alimentation (crédit). Exécute les débits et crédits après contrôle des fonds disponibles (aucun solde négatif). Enregistre chaque mouvement dans un journal et garantit l'idempotence des opérations répétées. Unique autorité d'écriture des soldes.	Spring Boot	8082
MS3 – Payment	Orchestre les paiements et transferts : crée le paiement, interroge User, Fraud, Card et Wallet, puis applique le verdict. Gère les statuts (PENDING, PENDING_REVIEW, COMPLETED, REJECTED, FAILED, CANCELLED), conserve l'historique des paiements, empêche les doublons et publie `payment.completed`.	Spring Boot	8083
MS4 – Card	Gère les cartes virtuelles : création, définition des plafonds, blocage et déblocage. Valide l'état et la validité d'une carte avant son utilisation. Bloque automatiquement les cartes d'un compte suspendu et publie `card.blocked`.	Spring Boot	8084
MS5 – Fraud	Évalue chaque transaction par des règles (montant inhabituel, fréquence anormale, appareil ou pays nouveau) et calcule un score de risque. Rend un verdict (APPROVED, SUSPICIOUS, REJECTED), crée les alertes, permet à l'analyste de les approuver ou rejeter, et publie `fraud.alert.created` et `fraud.review.completed`.	Spring Boot	8085

Composants d'infrastructure :

Composant	Rôle
API Gateway	Point d'entrée unique, routage
Keycloak	Identité, authentification, JWT / OAuth2
Eureka	Découverte de services
Config Server	Configuration centralisée
RabbitMQ	Événements asynchrones

3.3 Description détaillée de chaque service

MS1 – User

- **Rôle :** gérer le profil, le statut KYC, les rôles (CLIENT, ADMIN, FRAUD_ANALYST) et l'état du compte. Keycloak gère l'authentification et les JWT.
- **Entité :** User (id, keycloakId, firstName, lastName, email, phoneNumber, role, kycStatus, accountStatus, createdAt).
- **Endpoints :**
  - `GET /api/users/me` (200/401)
  - `PATCH /api/users/me` (200/400/401/409)
  - `GET /internal/users/{id}/eligibility` (200/404/503)
  - `PATCH /api/admin/users/{id}/status` (200/400/403/404/409)
  - `PATCH /api/admin/users/{id}/kyc` (200/400/403/404)
- **Interactions :** Wallet, Payment, Card et Fraud vérifient l'éligibilité du client en REST. User publie `user.suspended` et `user.reactivated`.
- **Panne :** si User est indisponible (503), les services qui exigent la vérification du client ne poursuivent pas l'opération.

MS2 – Wallet

- **Rôle :** posséder les soldes, appliquer les débits et crédits, et assurer l'unicité des opérations financières. Wallet est l'unique autorité d'écriture des soldes.
- **Entités :** Wallet (id, userId, balance, currency, status, version), WalletTransaction (id, walletId, operationId, paymentId, type, source, amount, balanceAfter, createdAt).
- **Endpoints :**
  - `POST /api/wallets` (201/400/401/409)
  - `GET /api/wallets/me` (200/401)
  - `GET /api/wallets/{id}/balance` (200/403/404)
  - `GET /api/wallets/{id}/transactions` (200/403/404)
  - `POST /internal/wallets/transfers` (200/400/409/422/503)

    - `GET /internal/wallets/{id}/eligibility` (200/404)
- **Interactions :** Payment appelle Wallet pour exécuter le transfert atomique, Card valide le portefeuille, Wallet vérifie l'éligibilité auprès de User. Wallet publie `wallet.transfer.completed` et consomme `user.suspended`.
- **Erreurs :** fonds insuffisants → 422. Opération répétée → rejet par contrôle d'idempotence (`operationId`).

MS3 – Payment

- **Rôle :** orchestrer les paiements et transferts, contrôler le risque, suivre les statuts et empêcher les doublons.
- **Entités :** Payment (id, userId, sourceWalletId, destinationWalletId, cardId, paymentType, amount, currency, status, fraudCheckId, idempotencyKey, failureReason, createdAt, completedAt), PaymentStatusHistory (id, paymentId, newStatus, reason, createdAt).
- **Endpoints :**
    - `POST /api/payments` (202/400/401/409/422)
    - `POST /api/transfers` (202/400/401/409/422)
    - `GET /api/payments` (200/401)
    - `GET /api/payments/{id}` (200/403/404)
    - `POST /api/payments/{id}/cancel` (200/403/404/409)
    - `GET /api/payments/{id}/receipt` (200/403/404/409)
- **Interactions :** REST vers User (éligibilité), Fraud (verdict), Card (validation de la carte) et Wallet (transfert). Payment publie `payment.completed` et `payment.failed`, et consomme `fraud.review.completed`.
- **Pannes :** Fraud indisponible → paiement non exécuté ou mis en attente, jamais approuvé par défaut. Transfert Wallet échoué → paiement `FAILED`.

MS4 – Card

- **Rôle :** gérer les cartes virtuelles simulées ou tokenisées, leur statut et leurs limites de dépense.
- **Entités :** Card (id, userId, walletId, cardToken, lastFourDigits, expiryMonth, expiryYear, spendingLimit, limitPeriod, status), CardStatusHistory (id, cardId, newStatus, reason, createdAt).
- **Endpoints :**
    - `POST /api/cards` (201/400/401/409)
    - `GET /api/cards` (200/401)
    - `GET /api/cards/{id}` (200/403/404)
    - `PATCH /api/cards/{id}/block` (200/403/404/409)
    - `PATCH /api/cards/{id}/unblock` (200/403/404/409)
    - `PUT /api/cards/{id}/limit` (200/400/403/404)
    - `POST /internal/cards/{id}/validate-payment` (200/400/404/422)
- **Interactions :** REST vers User (propriétaire) et Wallet (portefeuille associé), et appelé par Payment pour valider un paiement. Card publie `card.blocked` et consomme `user.suspended` pour bloquer les cartes actives du compte suspendu.
- **Erreurs :** carte déjà bloquée → 409. Messages en double traités de façon idempotente.

MS5 – Fraud Detection (conception commune de l'équipe)

- **Rôle :** évaluer chaque transaction par des règles simples (montant inhabituel, fréquence anormale, appareil ou pays nouveau), conserver un score et ses explications, générer des alertes et enregistrer les décisions de revue.
- **Entités :** FraudCheck (id, paymentId, userId, amount, riskScore, verdict, rulesTriggered, checkedAt), FraudAlert (id, fraudCheckId, alertType, status, decision, reason, reviewedBy, reviewedAt, createdAt).
- **Endpoints :**
  - `POST /internal/fraud/checks` (200/400/409/503)
  - `GET /api/admin/fraud/checks/{id}` (200/403/404)
  - `GET /api/admin/fraud/alerts` (200/403)
  - `GET /api/admin/fraud/alerts/{id}` (200/403/404)
  - `PATCH /api/admin/fraud/alerts/{id}/decision` (200/400/403/404/409)
- **Interactions :** appelé par Payment en REST synchrone, avec un contrôle optionnel de l'état du compte auprès de User. Fraud publie `fraud.alert.created` et `fraud.review.completed`, et consomme `payment.completed`.
- **Verdicts :** APPROVED, SUSPICIOUS, REJECTED. Un verdict SUSPICIOUS crée une alerte et place le paiement en `PENDING_REVIEW`

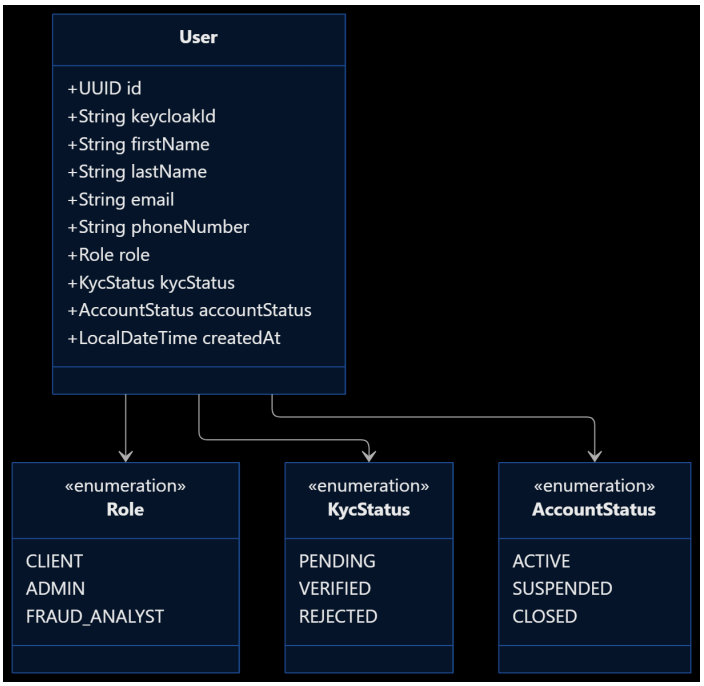
4. Données gérées par chaque service

4.1 Vue d'ensemble

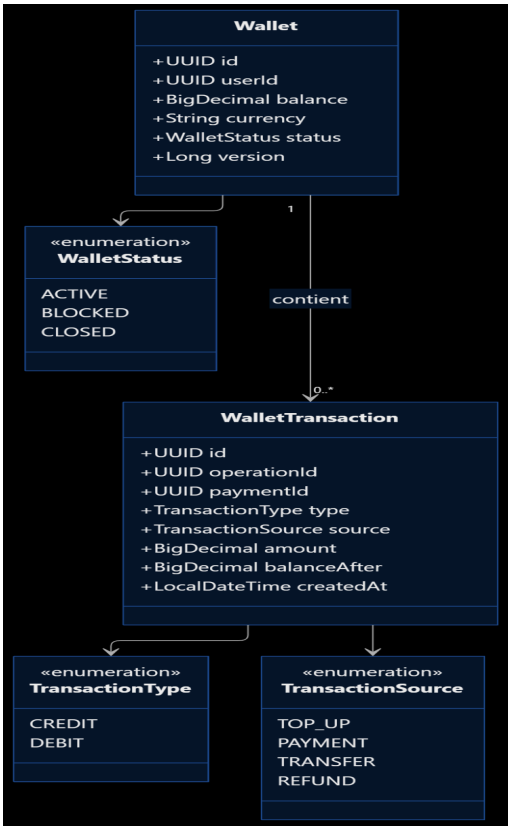
Microservice	Base de données	Type (SQL / NoSQL)	Entités principales
MS1 – User	H2	SQL (relationnelle)	users
MS2 – Wallet	H2	SQL (relationnelle)	wallets, wallet_transactions
MS3 – Payment	H2	SQL (relationnelle)	payments, payment_status_history
MS4 – Card	MySQL	SQL (relationnelle)	cards, card_status_history
MS5 – Fraud	PostgreSQL	SQL (relationnelle)	fraud_checks, fraud_alerts

4.2 Modèle de données par service

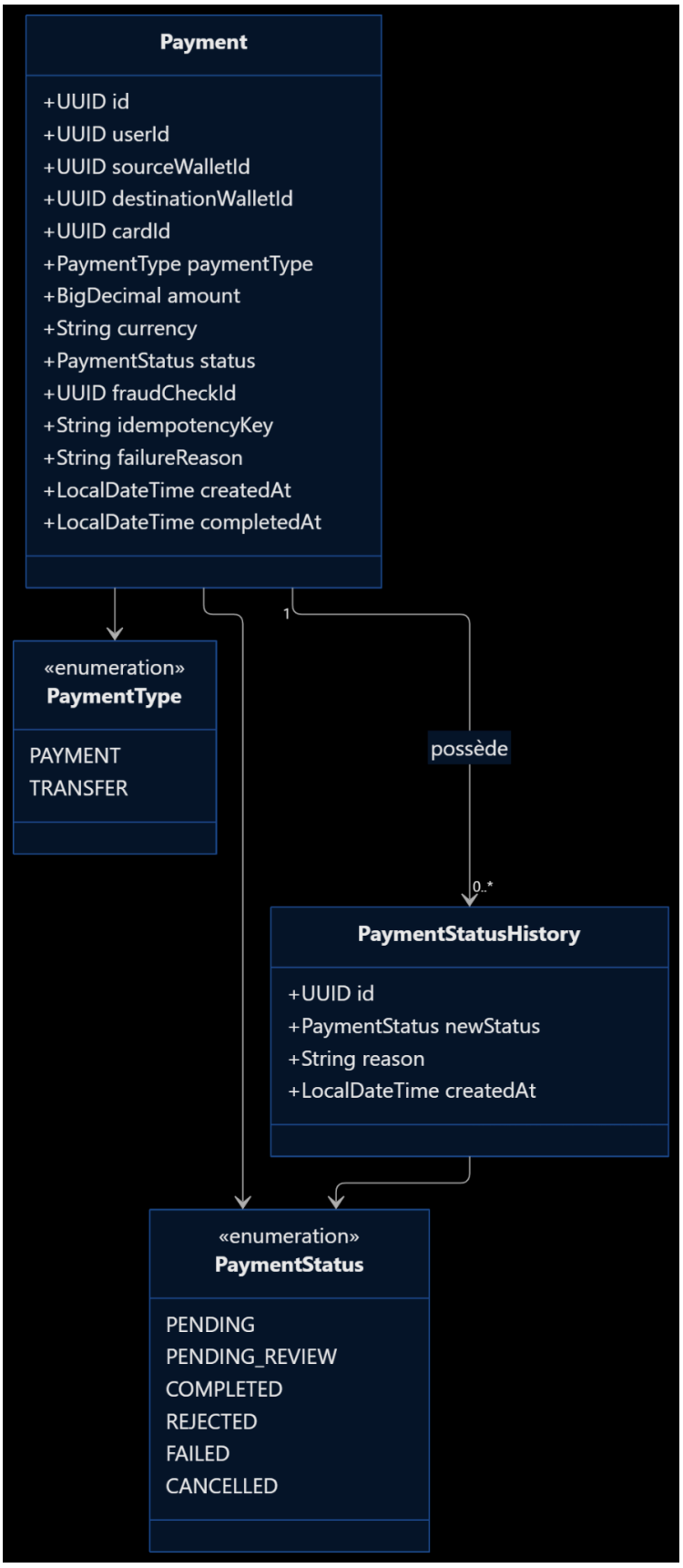
MS1 – User (DB_USER, H2)



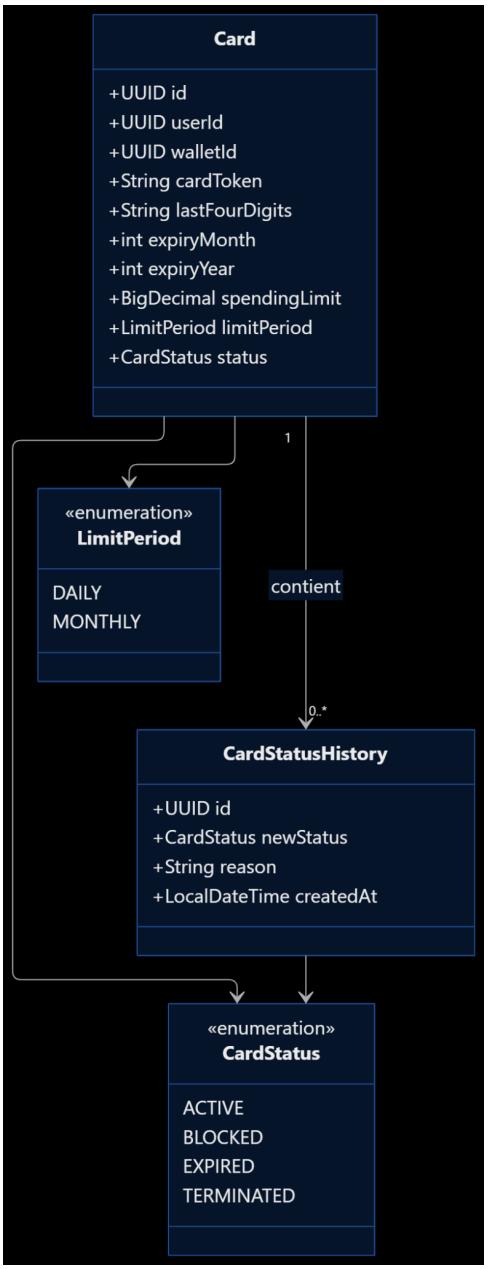
MS2 – Wallet (DB_WALLET, H2)



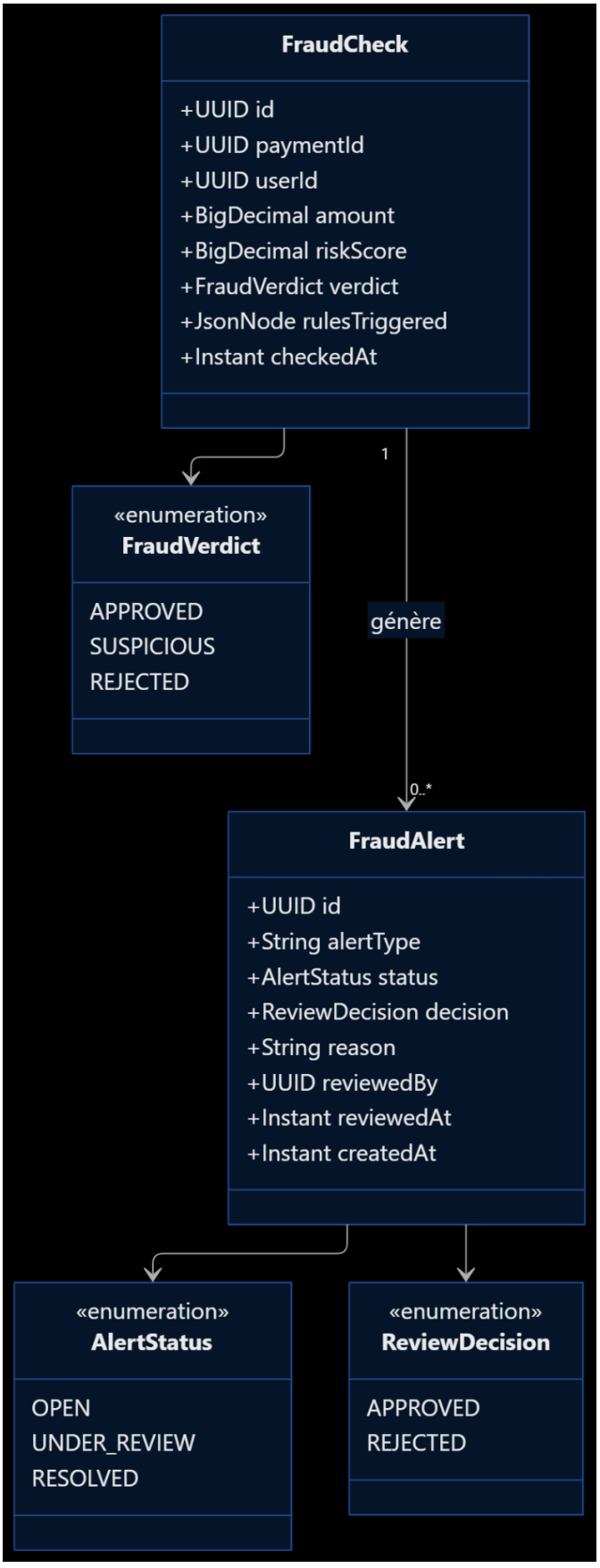
MS3 – Payment (DB_PAYMENT, H2)



MS4 – Card (DB_CARD, MySQL)



MS5 – Fraud Detection (DB_FRAUD, PostgreSQL)



4.3 Gestion de la cohérence des données

Chaque microservice possède sa propre base, donc il n'existe pas de transaction ACID globale. La cohérence est **forte à l'intérieur d'un service** et **éventuelle entre services**.

- **Références logiques :** `userId`, `walletId`, `cardId` et `paymentId` n'ont pas de clé étrangère inter-base. Leur validité est vérifiée par appel REST interne.
- **Autorité unique :** Wallet est le seul à écrire les soldes. User gère `accountStatus`, Card gère `CardStatus`, Payment gère `PaymentStatus` et Fraud gère les verdicts et les décisions.
- **Transactions locales :** un transfert interne débite et crédite deux portefeuilles dans une seule transaction SQL. Le champ `version` évite les écritures concurrentes et un solde négatif est refusé (422).
- **Idempotence :** `Idempotency-Key` pour les paiements, `operationId` pour Wallet, consommateurs RabbitMQ idempotents.
- **Traçabilité :** `PaymentStatusHistory`, `CardStatusHistory` et `WalletTransaction` (avec `balanceAfter`) conservent chaque changement.
- **Événements :** `user.suspended`, `card.blocked`, `fraud.review.completed` et `payment.completed` passent par RabbitMQ, avec une outbox transactionnelle pour fiabiliser la publication. Un compte suspendu (SUSPENDED) entraîne le blocage de ses portefeuilles et de ses cartes actives.
- **Compensation :** si un paiement échoue après un débit, Wallet recrédite le montant par une écriture `CREDIT` de source `REFUND`.

Situation	Traitement
Fraud indisponible	Jamais approuvé par défaut : paiement non exécuté ou échec temporaire (503).
Transaction suspecte	`PENDING_REVIEW`, alerte `OPEN`, aucun mouvement de fonds.
Approbation par l'analyste	**Payment revérifie l'état `PENDING_REVIEW` avant d'appeler Wallet.**
Échec du transfert Wallet	Transaction annulée en bloc, paiement `FAILED`.
Message en double	Ignoré par idempotence.

5. Interactions entre les services

5.1 Communications synchrones

Service source	Service cible	Endpoint	Objet de l'appel
Payment	User	GET /internal/users/{id}/eligibility	Vérifier le statut et le KYC du client
Payment	Fraud	POST /internal/fraud/checks	Obtenir le verdict avant tout mouvement de fonds
Payment	Card	POST /internal/cards/{id}/validate-payment	Contrôler le statut, l'expiration et le plafond
Payment	Wallet	POST /internal/wallets/transfers	Exécuter le transfert approuvé (atomique)
Wallet	User	GET /internal/users/{id}/eligibility	Vérifier l'éligibilité avant création ou recharge
Card	User	GET /internal/users/{id}/eligibility	Vérifier le propriétaire avant émission
Card	Wallet	GET /internal/wallets/{id}/eligibility	Valider le portefeuille et sa devise
Fraud	User	GET /internal/users/{id}/eligibility	Contrôle optionnel de l'état du compte
Analyste	Fraud	PATCH /api/admin/fraud/alerts/{id}/decision	Approuver ou rejeter une alerte

Les endpoints `/internal` sont authentifiés entre services et inaccessibles depuis Internet.

5.2 Communications asynchrones (RabbitMQ)

Événement	Producteur	Consommateur(s)	Description
user.suspended	User	Card, Wallet, Payment	Bloque les cartes et gèle les portefeuilles
user.reactivated	User	Card, Wallet	Réactivation, sans débloquer une carte déjà bloquée
wallet.transfer.completed	Wallet	Payment, reporting	Résultat d'un transfert exécuté
payment.completed	Payment	Fraud, reporting	Paiement finalisé (jamais une nouvelle instruction de débit)
payment.failed	Payment	Fraud, reporting	Échec d'un paiement

Événement	Producteur	Consommateur(s)	Description
fraud.alert.created	Fraud	Notification / suivi	Nouvelle alerte à examiner
fraud.review.completed	Fraud	Payment	Décision de l'analyste sur une transaction `PENDING_REVIEW`
card.blocked	Card	Payment, supervision	Carte bloquée

5.3 Scénarios d'interaction

Scénario 1 : transfert accepté

1. Le client appelle `POST /api/transfers` avec un JWT et une `Idempotency-Key`. Payment enregistre le paiement en `PENDING` (réponse 202).
2. Payment vérifie l'éligibilité auprès de User, puis la carte auprès de Card si besoin.
3. Payment demande le verdict à Fraud, qui répond `APPROVED`.
4. Payment appelle Wallet, qui débite et crédite dans une seule transaction SQL et publie `wallet.transfer.completed`.
5. Payment passe à `COMPLETED` et publie `payment.completed`.

Scénario 2 : transaction suspecte

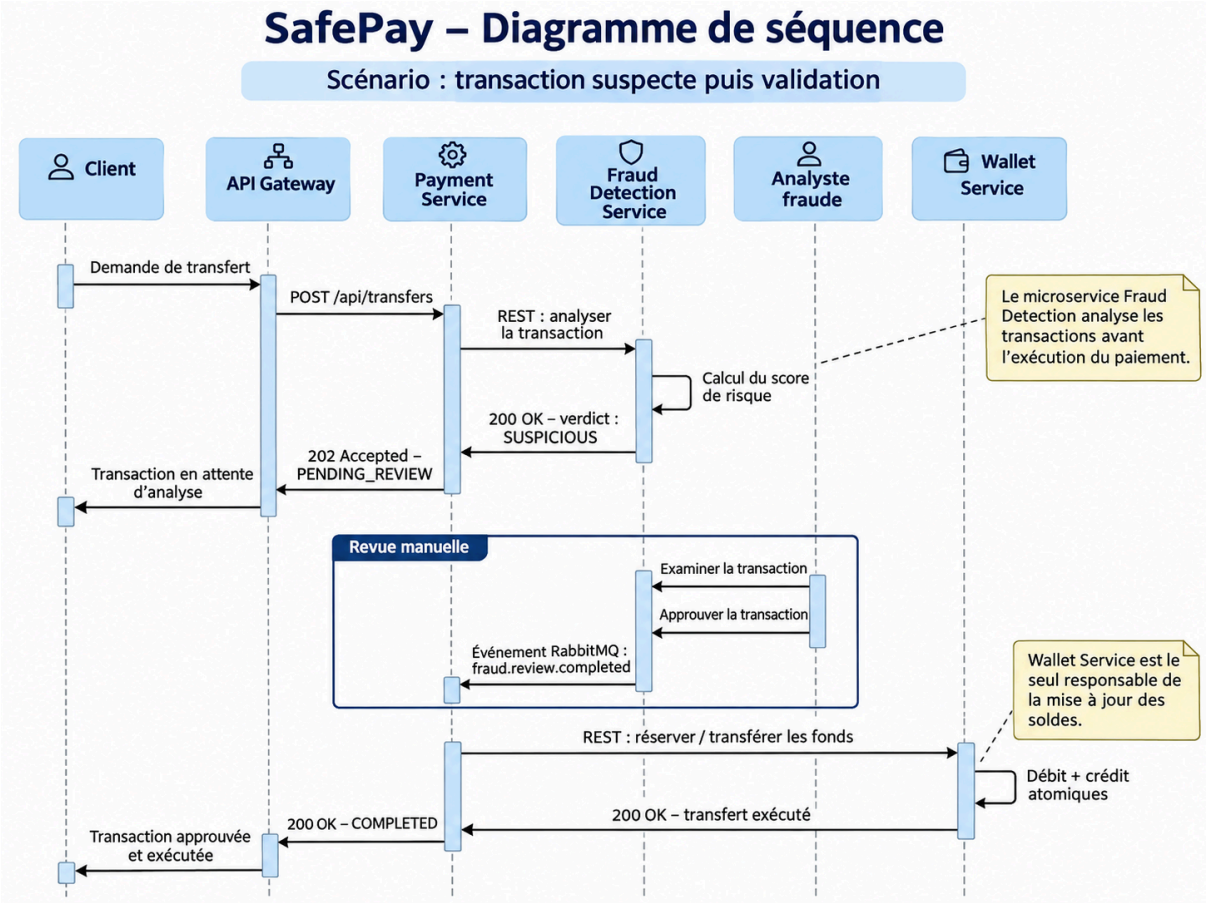
1. Fraud répond `SUSPICIOUS` et crée une alerte `OPEN`, puis publie `fraud.alert.created`.
2. Payment passe à `PENDING_REVIEW`, sans mouvement de fonds.
3. L'analyste examine l'alerte (`UNDER_REVIEW`) puis enregistre sa décision (`RESOLVED`).
4. Fraud publie `fraud.review.completed`. Payment vérifie que l'état est toujours `PENDING_REVIEW`, puis finalise (approbation) ou passe à `REJECTED`.

Scénario 3 : suspension d'un compte

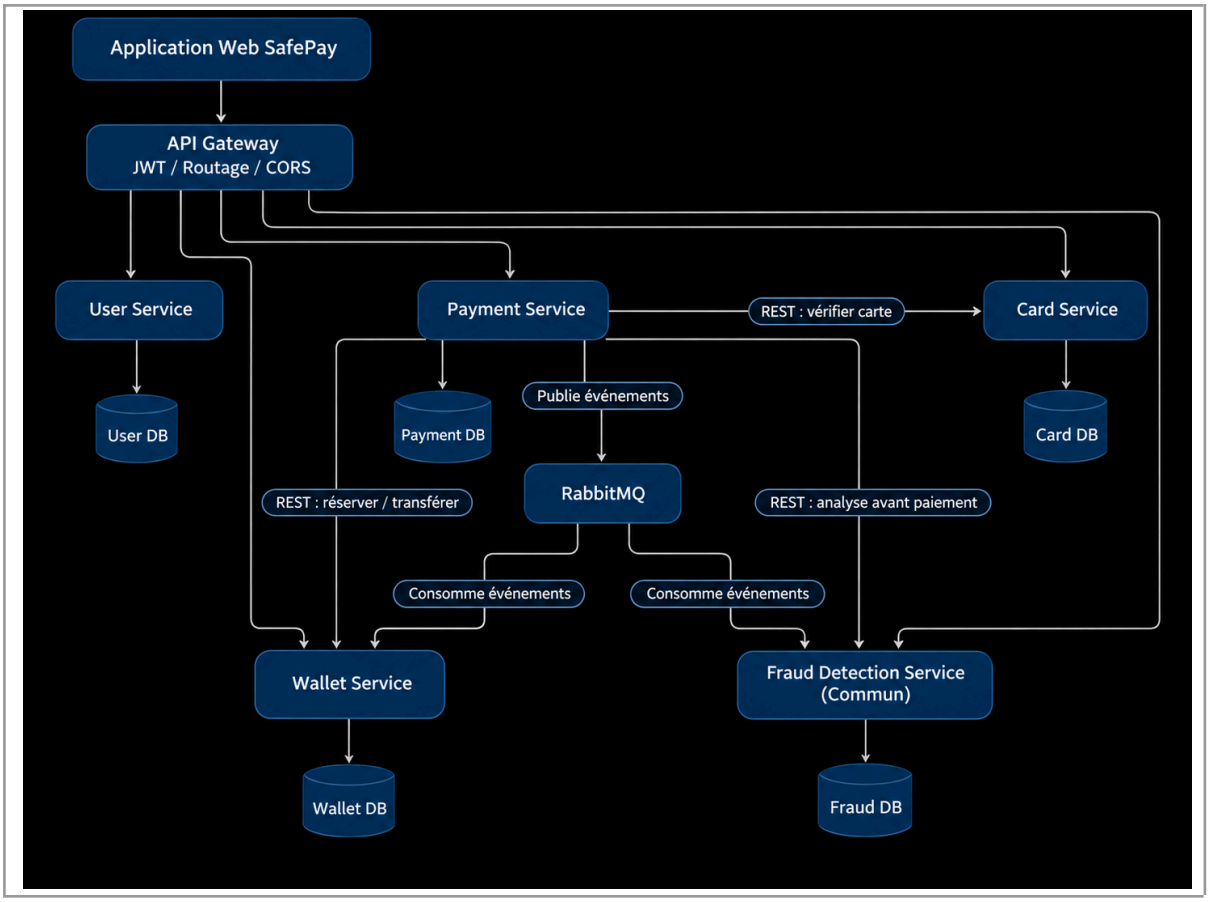
1. L'administrateur appelle `PATCH /api/admin/users/{id}/status`.
2. User met à jour `accountStatus` puis publie `user.suspended`.
3. Card bloque les cartes actives et publie `card.blocked`. Wallet gèle les portefeuilles.

Si Fraud est indisponible : le paiement n'est jamais approuvé par défaut. Il reste non exécuté (échec temporaire) ou en attente.

5.4 Diagramme de séquence



6. Diagramme d'architecture globale



Légende et explications

7. Autres éléments de conception

7.1 Technologies utilisées

Couche	Technologie	Justification
Backend	Spring Boot	Écosystème mature pour les microservices
Passerelle	API Gateway	Point d'entrée unique et routage
Découverte / configuration	Eureka, Config Server	Localisation des services, configuration centralisée
Communication	REST (OpenFeign), RabbitMQ	Réponses immédiates vs événements découplés
Sécurité	Keycloak (OIDC / JWT	Identités centralisées, le profil reste dans User
Bases de données	H2, MySQL, PostgreSQL	H2 pour la démonstration, MySQL relationnel pour Card, PostgreSQL (JSONB) pour les preuves antifraude

Couche	Technologie	Justification
Déploiement	Docker Compose	Lancement de tous les services ensemble
Documentation API	OpenAPI / Swagger	Contrats REST harmonisés

7.2 API Gateway et découverte de services

API Gateway. C'est le point d'entrée unique de la plateforme : le frontend (web ou mobile) ne contacte jamais directement un microservice. Elle route chaque requête vers le bon service selon le chemin (`/api/users/**` vers User, `/api/wallets/**` vers Wallet, `/api/payments/**` et `/api/transfers/**` vers Payment, `/api/cards/**` vers Card, `/api/admin/fraud/**` vers Fraud). Elle vérifie le JWT émis par Keycloak et rejette les requêtes non authentifiées (401). Les routes `/internal/**` ne sont pas exposées : elles ne servent qu'aux appels entre services.

Découverte de services (Eureka). Au démarrage, chaque service s'enregistre auprès d'Eureka sous son nom (par exemple `payment-service`). La Gateway et les appels OpenFeign retrouvent ainsi l'adresse d'un service par son nom, sans adresse fixe, ce qui convient à des conteneurs dont l'adresse change. Eureka retire les instances qui ne répondent plus.

Configuration centralisée (Config Server). Les paramètres des services (ports, bases de données, RabbitMQ, Keycloak) sont regroupés à un seul endroit et récupérés au démarrage, ce qui évite de les dupliquer dans chaque projet.

7.3 Sécurité

Keycloak authentifie les utilisateurs (OIDC) et émet les JWT, tandis que User Service gère le profil et l'éligibilité métier (KYC, compte actif). Les endpoints `/api/admin` sont réservés aux rôles ADMIN et FRAUD_ANALYST, et les endpoints `/internal` sont authentifiés entre services. Les cartes sont simulées ou tokenisées : seuls `cardToken` et `lastFourDigits` sont stockés, jamais le numéro complet. Les mots de passe relèvent uniquement de Keycloak.

7.4 Déploiement et conteneurisation

Chaque microservice est conteneurisé avec **Docker** (un `Dockerfile` par service). L'ensemble est lancé avec **Docker Compose** : les cinq services, leurs bases (MySQL et PostgreSQL ; H2 est embarquée dans le service), RabbitMQ, Keycloak, Eureka, Config Server et l'API Gateway. Une seule commande démarre toute la plateforme, ce qui facilite la démonstration et les tests d'intégration. Le dépôt Git est organisé en `Backend/` (un dossier par service), `Frontend/` et `Documentation/`. Kubernetes et une chaîne CI/CD ne sont pas prévus dans ce projet mais constituent la suite logique en production.

7.5 Supervision et journalisation

- **Logs par service :** chaque microservice écrit ses journaux, avec les identifiants utiles pour suivre une opération (`paymentId`, `operationId`, `eventId`).

- **Traçabilité métier :** `PaymentStatusHistory`, `CardStatusHistory` et `WalletTransaction` conservent chaque changement d'état, ce qui permet de reconstituer le déroulement d'un paiement.
- **Suivi de l'état des services :** Eureka indique quelles instances sont actives, et les endpoints de santé (Spring Boot Actuator) permettent de vérifier chaque service.
- **Évolution possible :** centralisation des logs et métriques (ELK, Prometheus, Grafana) et traçage distribué.

7.6 Gestion des erreurs et résilience

- **Timeouts et circuit breaker :** les appels REST vers Fraud, User, Card et Wallet ont un délai maximal. Si une dépendance ne répond pas, le circuit breaker évite de la saturer et renvoie rapidement une erreur 503.
- **Repli prudent :** si Fraud est indisponible, le paiement n'est jamais approuvé par défaut. Il reste non exécuté ou en attente.
- **Retry :** les nouvelles tentatives sont possibles car les opérations sont idempotentes (`Idempotency-Key`, `operationId`) : un rejeu ne crée pas de doublon.
- **Messages fiables :** les consommateurs RabbitMQ sont idempotents et les événements sont publiés avec une outbox transactionnelle.
- **Erreurs métier explicites :** 409 pour un conflit d'état ou un doublon, 422 pour fonds insuffisants ou plafond dépassé.
- **Compensation :** en cas d'échec après un débit, Wallet recrédite le montant (écriture `CREDIT` de source `REFUND`).

7.7 Limites et améliorations futures

Limites du prototype

- H2 convient à la démonstration, mais pas à un paiement réel.
- Les données de paiement et les cartes sont simulées, sans intégration bancaire.
- La détection de fraude repose sur des règles simples.
- La cohérence entre services est éventuelle, pas transactionnelle.

Améliorations possibles

- Bases durables et auditables, avec une comptabilité plus complète (écritures en partie double).
- Détection de fraude par machine learning.
- Supervision centralisée (ELK, Prometheus, Grafana).
- Orchestration avec Kubernetes et pipeline CI/CD.
- Notifications aux clients (e-mail ou SMS) à partir des événements d'alerte.

8. Conclusion

Ce document a présenté la conception de SafePay, une plateforme de paiement numérique sécurisée reposant sur une architecture en microservices. Le système est découpé en cinq services autonomes, User, Wallet, Payment, Card et Fraud Detection, chacun responsable d'une capacité métier précise et propriétaire de sa base de données. Les acteurs, les fonctionnalités, les données, les interactions et l'architecture globale ont été définis de manière cohérente.

Plusieurs choix de conception garantissent la fiabilité et la sécurité de la plateforme. Les échanges synchrones en REST servent aux décisions indispensables avant un débit, tandis que RabbitMQ découple les notifications et la reprise des revues manuelles. Fraud est consulté avant tout mouvement de fonds, Wallet est l'unique autorité sur les soldes, et l'idempotence ainsi que l'historique des statuts protègent contre les doublons et assurent la traçabilité. Keycloak centralise l'authentification, et l'API Gateway constitue l'unique point d'entrée.

Cette conception constitue une base solide pour la phase de développement : mise en place de la Gateway et des appels synchrones, puis des événements et de la sécurité, et enfin dockerisation et intégration. Elle pourra évoluer vers des bases durables, une détection de fraude par apprentissage automatique et une supervision centralisée, afin de s'approcher d'un système de paiement réel.